ARMORIQ x OPENCLAW HACKATHON

The Context

Autonomous AI agents are entering financial workflows. With frameworks like OpenClaw, agents can research equities, analyze earnings, monitor portfolios, and execute trades, operating directly on a user's local system without waiting for continuous prompts.

This unlocks real capability. It also introduces real risk.

An agent told to "look into NVDA and handle it" might research the stock, place an unauthorized buy, or silently forward portfolio data to an external endpoint. Each interpretation carries different consequences, and in financial systems, consequences are measured in dollars, compliance violations, and irreversible transactions.

These are not hypothetical risks. Security researchers at Microsoft, Cisco, and independent teams have documented fundamental vulnerabilities in the OpenClaw ecosystem: prompt injection through untrusted content, unauthorized tool execution, credential exposure, and silent scope escalation. For financial use cases, these risks are compounded by regulatory requirements, fiduciary responsibility, and the irreversibility of executed trades.

The capability exists. The guardrails do not.

In financial systems, intent must be enforced, not inferred.

The Core Question

How do we build autonomous OpenClaw agents that operate in financial workflows with guaranteed adherence to user-defined intent and constraints, even when the agent encounters ambiguous instructions, malicious inputs, or unexpected execution paths?

Your Challenge

Build an OpenClaw-based autonomous system that operates in a simulated financial environment using ArmorClaw for intent enforcement. Your system must:

- Perform meaningful multi-step reasoning across financial data
- Execute real actions against a live paper trading API (no mocked responses)
- Enforce clear intent boundaries at runtime using structured, interpretable policy models
- Demonstrate deterministic blocking of unauthorized behavior

All trading must use simulated funds via paper trading APIs such as Alpaca, TradeStation SIM, or equivalent simulators that provide realistic execution against live market data. No real money is involved at any stage.

Systems may interact with trading APIs, local data files, or broader financial workflows as needed.

The goal is not to build the most profitable trading bot.

The goal is to demonstrate intent-aware execution in a financial context.

Your system should show:

- Clear separation between reasoning and execution
- Explicit validation of intent before any financial action
- Deterministic enforcement of constraints on trades, data access, and tool usage
- Observable, autonomous blocking when rules are violated, without human intervention

Delegation scenarios are optional but will receive bonus consideration.

Example Directions

These are illustrative, not restrictive.

A stock analysis agent (single or multi-agent) that researches equities:

- Can: query market data, generate analysis reports, place trades within defined limits
- Must: log every trade decision with the intent it was validated against
- Cannot: exceed per-order or daily size limits, trade outside an approved ticker universe, access credential files or API keys

A portfolio monitoring agent that tracks holdings and generates alerts:

- Can: read portfolio positions, compute drift metrics, send alerts to approved channels
- Must: enforce read-only access to brokerage accounts at the policy level
- Cannot: execute any trades autonomously, transmit portfolio data to external endpoints, modify alert thresholds beyond user-defined bounds

An earnings research agent that processes financial reports:

- Can: fetch and summarize earnings filings, flag anomalies, write reports to a designated output directory
- Must: enforce time-based blackout windows that block all trade execution around earnings announcements
- Cannot: access documents outside its designated data directory, publish or share research outside approved channels

A personal finance agent that tracks spending and budgets:

- Can: parse transaction exports, categorize spending, generate budget summaries
- Must: restrict all file writes to a designated output folder, enforced at the policy level
- Cannot: access accounts beyond those explicitly authorized, initiate transfers or payments, expose transaction history to external services

A multi-agent advisory system with delegated authority:

- Can: research and generate trade recommendations (analyst), validate against portfolio limits (risk agent), execute approved recommendations (trader)
- Must: enforce bounded delegation where each agent operates only within its granted scope, with no implicit authority inheritance
- Cannot: allow any agent to exceed its delegated authority, access another agent's data, or sub-delegate without explicit policy approval

A compliance monitoring agent that audits trading activity:

- Can: read trade logs, flag rule violations (position limits, restricted lists, wash sale windows), generate structured audit reports
- Must: operate in strictly read-only mode against all trade records, enforced programmatically
- Cannot: modify trade records, override enforcement decisions, or suppress flagged violations

Your system may target any financial scenario, as long as it demonstrates intent-aware enforcement using simulated funds.

RULE BOOK

Team Structure

- Teams of 2 to 4 members
- All implementation must be completed during the hackathon
- Pre-built libraries, frameworks, and APIs are allowed (including ArmorClaw, Alpaca SDK, etc.)
- Fully pre-built agent systems are not allowed

Technical Requirements

Every submission must include:

- An OpenClaw-based autonomous agent
- Real execution of actions against a live paper trading API (no mocked responses)
- An intent validation layer before execution

- Policy-based runtime enforcement

Pure chatbot demos without execution will not qualify. Demos using real money will not qualify.

Architectural Expectations

Submissions must clearly demonstrate:

- Separation between reasoning and execution
- A visible enforcement layer
- Logging or traceability of decision-making

The system must show:

- At least one allowed financial action (e.g., a paper trade placed within policy)
- At least one blocked financial action (e.g., a trade rejected for exceeding limits, or a data exfiltration attempt stopped)
- Clear reasoning for why the action was allowed or blocked

Intent and Policy Design

Each team must define:

- A structured intent model
- A policy model with enforceable constraints

Financial constraint examples:

- Trade size limits (per order and daily aggregate)
- Ticker or asset class restrictions
- Directory-scoped file access for market data and reports
- Tool restrictions (e.g., no shell execution, no unauthorized uploads)
- Time-based restrictions (e.g., no trading outside market hours or during earnings blackouts)
- Spend or exposure limits
- Data handling restrictions (e.g., no PII or account numbers in tool arguments)

Intent and policy models must be structured and interpretable, not simple if-else checks. Submissions relying on hardcoded conditional logic without a declarative intent or policy representation will not be considered sufficient.

Enforcement must be programmatic and autonomous. Manual intervention or human-in-the-loop approval during execution is not permitted.

Delegation Bonus

Teams that implement bounded delegation will receive bonus points.

A valid delegation scenario must demonstrate:

- Limited scope authority (e.g., an analyst delegates a buy recommendation to a trader agent with a capped quantity and specific ticker)
- Explicit constraints on the delegated agent
- Blocking of attempts to exceed granted authority

Delegation is not required, but high-quality implementations will be rewarded.

Judging Criteria

Projects will be evaluated on:

A. Enforcement Strength

- Are constraints technically enforced at runtime?
- Are violations deterministically blocked without human intervention?

B. Architectural Clarity

- Is reasoning clearly separated from execution?
- Is the enforcement layer explicit and well designed?

C. OpenClaw Integration

- Does the system meaningfully leverage OpenClaw capabilities?

D. Accurate Delegation Enforcement

- If implemented, does the delegation mechanism correctly enforce scope boundaries?

E. Use Case Depth

- Is the financial scenario realistic and thoughtfully designed?
- Does it reflect genuine risks in autonomous financial workflows (e.g., unauthorized trades, data exfiltration, scope escalation)?
- Are enforcement challenges non-trivial, not just simple permission checks?

Submission Requirements

Each team must submit:

- Source code repository
- Architecture diagram
- Short document describing:

    - Intent model
    - Policy model
    - Enforcement mechanism
- Three-minute demo video demonstrating:
    - System overview
    - Allowed action
    - Blocked action
    - Explanation of enforcement

Finalist teams will present live with Q&A.

Resources

The following are optional references to help teams get started. They are not required reading.

Core Frameworks

- OpenClaw: Open-source autonomous AI agent framework
- OpenClaw Documentation: Official docs covering tools, skills, and security
- OpenClaw Skills Overview: How skills work and how to create them
- ClawHub: Central skill registry for OpenClaw

Intent Enforcement

- ArmorClaw: Intent enforcement plugin for OpenClaw agents
- ArmorIQ OpenClaw Docs: Setup, concepts, and configuration
- ArmorIQ: Intent Intelligence platform for AI agent security

Paper Trading APIs

- Alpaca Paper Trading: Free simulated trading with real-time market data
- Alpaca Trading API: Order execution, positions, and market data
- Alpaca MCP Server: Official MCP server for natural-language trading
- OpenClaw Alpaca Trading Skill: Community-built OpenClaw skill for Alpaca
- TradeStation SIM API: Alternative paper trading API

Security Background

- Microsoft: Running OpenClaw Safely: Identity, isolation, and runtime risk
- Cisco: Personal AI Agents Are a Security Nightmare: Enterprise risk analysis
- ClawJacked Vulnerability Disclosure: Agent reasoning hijack via malicious websites
- Bitdefender AI Skills Checker: Free security scanner for OpenClaw skills